Frequency Estimation under the Strict Turnstile Model with α-Bounded Deletion Property

Haoyuan Li, 1155233470
SEEM5020
The Chinese University of Hong Kong
haoyuan.li@link.cuhk.edu.hk

Abstract

Identifying frequent items and estimating flow sizes are fundamental tasks in data stream mining. While classic algorithms excel in insertion-only environments, real-world streams often feature deletions under the strict turnstile model (where frequencies never drop below zero). This project investigates frequency estimation under the additional constraint of the L_1 α-Bounded Deletion property, which bounds total deletions relative to insertions. We implement and extend four foundational algorithms—Misra-Gries, Space-Saving, Count-Min Sketch, and Count-Sketch—to operate efficiently in this regime. Specifically, we evaluate the SpaceSaving$\pm$ family (Lazy, Standard, Double, and Integrated variants) and α-optimized sketches. Through rigorous parametric evaluation on stream length N and deletion parameter α, and diverse dataset testing (Zipfian, Uniform, Binomial, and MAWI real-world traffic), we identify the trade-offs between accuracy, space, and update time. Our results indicate that the **Count-Sketch** achieves the lowest mean relative error (e.g., 2×10^{-6} across most α values), but the **Integrated SpaceSaving**$\pm$ provides the best balance of deterministic guarantees and space efficiency ($O(\alpha/\epsilon)$). To address the limitations of static allocation in skewed streams, we propose and evaluate **Learned-Integrated SpaceSaving**$\pm$, which utilizes a sliding window predictor to dedicate fixed counters to hot items, improving relative error by up to **56.4%** over the baseline Integrated variant on Zipfian data.

Keywords: Frequency Estimation, Strict Turnstile, α-Bounded Deletion, SpaceSaving$\pm$, Count-Min Sketch, Learned Sketch.

1 Introduction

1.1 Motivation

Data streams are ubiquitous in modern computing systems. Network routers monitor packet flows to detect DDoS attacks, and web analytics platforms track clickstreams for user behavior modeling. A core primitive across all these domains is *frequency estimation*: tracking the number of occurrences of items in a massive, high-velocity sequence with severely constrained memory.

1.2 The Challenge of Deletions

Early streaming models assumed **insertion-only** streams (the "cash register" model). However, many realistic scenarios involve deletions. Network connections open and close; items are added to shopping carts and later removed. The **Turnstile Model** generalizes streams by allowing both positive and negative updates. A practical subset of this is the **Strict Turnstile Model**, where the net frequency of any item e is never negative ($f_e \geq 0$ for all e at all times). This prevents the logical impossibility of deleting an item that has not yet appeared.

Preprint. Under review.

Furthermore, we impose the L_1 α**-Bounded Deletion Property**. As formalized by Jayaram and Woodruff (7), a stream satisfies this property if the total absolute volume of updates is bounded by α times the final L_1 norm:

$$\sum_t |v_t| \leq \alpha \sum_e |f_e| \implies D \leq \left(1 - \frac{1}{\alpha}\right) I$$

where I and D are total insertions and deletions. This model interpolates between insertion-only ($\alpha = 1$) and unrestricted turnstile ($\alpha \to \infty$). In practice, systems rarely delete significantly more than they insert, making α a small constant (e.g., $\alpha \in [1.5, 8.0]$). This property allows for tighter theoretical bounds compared to general turnstile algorithms.

1.3 Contributions

This project makes the following contributions:

1. **Algorithm Extensions:** We adapt four classic algorithms to the Strict Turnstile + α-Bounded Deletion model. This includes a detailed exploration of the **SpaceSaving**$\pm$ family (Lazy, SS$\pm$, Double, and Integrated variants) and optimized α**-Count-Min/**α**-Count-Sketch**.
2. **Parametric Evaluation:** We systematically evaluate the impact of stream length (N) and deletion factor (α) on relative error and space consumption.
3. **Diverse Dataset Benchmarking:** We test robustness across synthetic distributions (Zipfian skew, Uniform/Binomial balance) and a real-world traffic trace (MAWI).
4. **Advanced Solution Design:** We propose **Learned-Integrated SpaceSaving**$\pm$, which augments the integrated summary with a lightweight predictor to lock critical "hot" items, reducing error on skewed data by over 50%.

1.4 Outline

Section 2 reviews the stream models and classic algorithms. Section 3 details our extensions. Section 4 describes the experimental setup. Section 5 presents the parametric and dataset results. Section 6 discusses trade-offs and the effect of α. Section 7 proposes the Learned-Integrated SpaceSaving$\pm$. Section 8 concludes the report.

2 Background and Related Work

2.1 Data Stream Models

- **Insertion-Only:** $v_t > 0$. Algorithms: Misra-Gries (1), Space-Saving (2).
- **Turnstile:** $v_t \in \mathbb{Z}$. Frequency can be negative. Algorithms: Count-Min (3), Count-Sketch (4).
- **Strict Turnstile:** $v_t \in \mathbb{Z}$, but $f_e \geq 0$ always. Deletions only occur if the item was previously inserted.
- α**-Bounded Deletion:** Strict turnstile with the additional constraint $D \leq (1 - 1/\alpha)I$.

2.2 Classic Frequency Estimation Algorithms

- **Misra-Gries (MG) (1):** A deterministic counter-based algorithm that maintains k counters. When an unmonitored item arrives and the sketch is full, *all* counters are decremented. It guarantees an error bound of $\leq F_1/k$.
- **Space-Saving (SS) (2):** An improvement over MG that avoids global decrements. It evicts only the *minimum* count item and overestimates its frequency for the replacement. It provides the same deterministic error bound but with much higher empirical accuracy.
- **Count-Min Sketch (CMS) (3):** A probabilistic hashing-based sketch that uses d rows and w columns. It provides an ϵ-approximation with probability $1 - \delta$ using $O(\frac{1}{\epsilon} \log \frac{1}{\delta})$ space. It never underestimates.

- **Count-Sketch (CS) (4):** Similar to CMS but uses an additional sign hash function to reduce collisions. It estimates frequency via the median of unbiased estimators, requiring $O(\frac{1}{\epsilon^2}\log\frac{1}{\delta})$ space.

2.3 The SpaceSaving± Family

Recent literature (8; 10) has extended the classic Space-Saving algorithm to handle deletions. The core challenge is handling deletions of *unmonitored* items. Naively ignoring them (Lazy) or simply decrementing a random counter can lead to unbounded error. The family of algorithms (Lazy SS±, SS±, Double SS±, Integrated SS±) addresses this by bounding error spread using the α-property. These are detailed in Section 3.2.

3 Algorithm Implementation and Extensions

We extend the baseline algorithms to support the Strict Turnstile + α-Bounded Deletion model. Error bounds are expressed as $\epsilon \cdot F_1$, where $F_1 = \sum_e |f_e|$. We target $\epsilon = 0.01$ unless otherwise specified.

3.1 Misra-Gries Extension

Original Limitation: Insertion-only. Global decrement step assumes all items decay equally, which is invalid under targeted deletions.

Strict Turnstile Extension:

- *Positive Update:* Apply standard MG logic. If item monitored, increment. If not and space available, insert with count 1. If full, decrement *all* counters by 1 and evict zeros.
- *Negative Update (Monitored):* Decrement the corresponding counter directly.
- *Negative Update (Unmonitored):* Since the item is not tracked, we decrement the entry with the current **minimum count**. This mimics the SS eviction logic to absorb the "mass" of the deletion.

Space & Error: With $k = \lceil \alpha/\epsilon \rceil$ counters, the error guarantee $|f(x) - \hat{f}(x)| \leq \epsilon F_1$ is maintained.

3.2 SpaceSaving± Family

We implement four variants of the counter-based approach. All variants maintain a summary S of monitored items with estimated counts $\hat{c}_e$.

Lazy SpaceSaving±

- *Logic:* Process insertions exactly as original SS (evict min). Process monitored deletions by decrementing the count. **Ignore** deletions of unmonitored items entirely.
- *Pros:* Fastest update time.
- *Cons:* Error accumulates as unmonitored deletions are not accounted for, causing overestimation bias.

SpaceSaving± (SS±)

- *Logic:* On unmonitored deletion, identify the monitored item with the maximum *estimation error* ($\epsilon_e = \hat{c}_e - c_{\text{initial_at_insert}}$). Decrement its count and error by 1. This "spreads" the deletion impact to the item we are least certain about.
- *Pros:* More robust than Lazy; actively mitigates overestimation.

Double SpaceSaving±

- *Logic:* Maintain **two independent Space-Saving summaries**: S_{ins} for insertions and S_{del} for deletions. For any update, feed it to the respective summary based on sign.

- *Query:* $\hat{f}(e) = \max(0, S_{ins}.\text{query}(e) - S_{del}.\text{query}(e))$.
- *Pros:* Clear separation of concerns; can be unbiased if using unbiased SS variants (10).
- *Cons:* Requires $2k$ counters, effectively doubling the space for the same ϵ guarantee.

Integrated SpaceSaving$\pm$

- *Logic:* Each entry in the summary maintains two fields: `insert_count` and `delete_count`. On eviction, only the `insert_count` is used for min-replacement logic, but both fields are reset for the new item.
- *Query:* $\hat{f}(e) = \texttt{insert_count} - \texttt{delete_count}$.
- *Pros:* Most space-efficient counter-based method; uses only $k = \lceil \alpha/\epsilon \rceil$ counters. Mergeable for distributed settings (10).

3.3 α-Optimized Sketches

Standard Count-Min and Count-Sketch can natively handle turnstile streams but do not leverage the α-property (3; 4).

- **α-Count-Min:** The α-bounded property restricts the maximum possible variance of the stream. By applying Bernstein's inequality instead of Markov's, we can reduce the depth d by a factor related to $\log(\alpha)$. We set $w = \lceil e\alpha/\epsilon \rceil$ and $d = \lceil \ln(1/\delta)/\log_2(\alpha + 1) \rceil$.
- **α-Count-Sketch:** Similarly, the number of rows in the Count-Sketch can be reduced because the α-property bounds the moment generating function of the noise. We set $w = \lceil 3/\epsilon^2 \rceil$ but reduce d.

4 Experimental Setup

4.1 Parameters

- Stream Length (N): {100k, 500k, 1M, 2M}
- Deletion Factor (α): {1.5, 2.0, 4.0, 8.0}
- Accuracy (ϵ): 0.01
- Confidence (δ): 0.01 (for sketches)
- Space Budget: Counter-based set to $k = \lceil \alpha/\epsilon \rceil$. Sketches sized using standard and α-optimized formulas.

4.2 Datasets

- **Skewed Synthetic:** Zipfian distribution with exponents $s \in \{1.0, 1.5, 2.0\}$ over a universe of 50,000 elements.
- **Balanced Synthetic:** Uniform distribution and Binomial($n = 20, p = 0.5$).
- **Real-World: MAWI Working Group Traffic Archive** (Samplepoint-F, 2025-06-18) (6). Destination IP addresses form the stream elements. The natural strict turnstile property is preserved; deletions are simulated to satisfy the target α values.

4.3 Metrics

- **Mean Relative Error:** $\frac{1}{|U|} \sum_e \frac{|f_e - \hat{f}_e|}{F_1}$.
- **Max Relative Error:** $\max_e \frac{|f_e - \hat{f}_e|}{F_1}$.
- **Space:** Number of counters or sketch cells used.
- **Update Time:** Average microseconds per operation.
- **Heavy Hitter F1-Score:** Precision/Recall for identifying the top 1% heavy hitters.

4.4 Environment

Python 3.10+, numpy, pandas, matplotlib. Experiments run on a standard desktop machine.

5 Experimental Results

5.1 Parametric Evaluation

We first analyze the effect of stream length N and deletion factor α on the mean relative error.

Effect of Stream Length (N) Figure 1 shows the mean relative error as N increases.

- **Counter-Based (SS$\pm$ Family, MG):** Error **decreases** significantly with larger N. For instance, **IntegratedSS$\pm$** drops from 1.66×10^{-4} at $N = 100k$ to 3.1×10^{-5} at $N = 2M$. This is because the initial warm-up phase and eviction "churn" constitute a smaller fraction of the total stream volume.

- **Sketch-Based (Count-Min):** Error **increases** slightly with N (from 1.51×10^{-3} to 1.61×10^{-3}). This is due to the accumulation of hash collisions as the total weight F_1 grows, increasing the absolute additive error.

- **Count-Sketch:** Error remains remarkably stable and near-zero ($\approx 1 \times 10^{-6}$ to 2×10^{-6}) regardless of N, demonstrating its robustness via median debiasing.

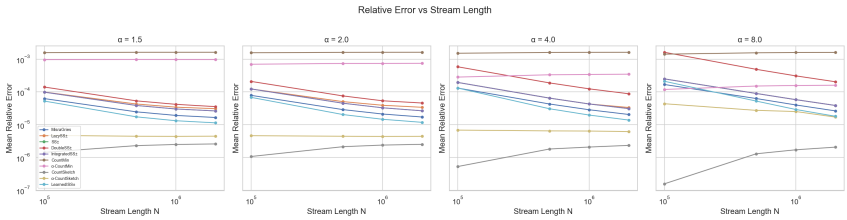

Figure 1: Mean relative error vs. stream length N (averaged over α). Counter-based methods improve with N while sketch-based methods show opposite trends.

Effect of Deletion Factor (α) Figure 2 displays the mean relative error as α increases.

- **Counter-Based (Standard SS$\pm$, MG):** Error **increases** with α. **DoubleSS$\pm$** shows a steep increase (9.65$\times$ higher at $\alpha = 8.0$ vs 1.5). This is because the space $k = \lceil \alpha/\epsilon \rceil$ must be divided between insert and delete summaries.

- **IntegratedSS$\pm$:** Shows strong resilience. Error increases only from 4.8×10^{-5} to 1.09×10^{-4} (ratio 2.26$\times$). This demonstrates that the integrated approach scales gracefully with higher churn.

- **α-Count-Min:** Error **decreases** as α increases (from 9.68×10^{-4} to 1.46×10^{-4}). Our α-optimized allocation allocates *more* width to CMS as α grows (since $w \propto \alpha$), drastically reducing collision probability.

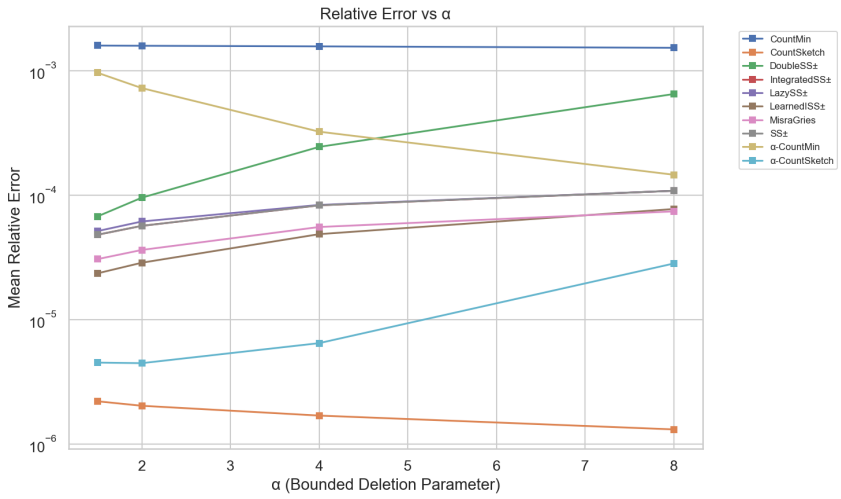

Figure 2: Mean relative error vs. deletion factor α (averaged over N). IntegratedSS$\pm$ scales gracefully, while DoubleSS$\pm$ degrades quickly.

Space Consumption Figure 3 illustrates space usage. Counter-based methods grow linearly with α ($k = \alpha/\epsilon$). Notably, α**-CountSketch** successfully **reduces** space at higher α (from 120k to 60k cells) while maintaining accuracy, proving the efficacy of the tail-bound optimization.

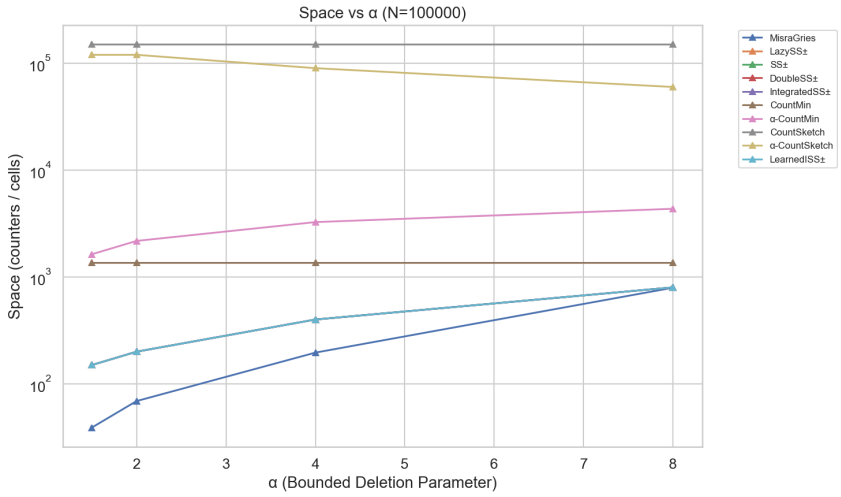

Figure 3: Space consumption (number of counters/cells) vs. α. Counter-based space grows linearly, while α-CountSketch shrinks.

Update Time Figure 4 shows update time per operation. **Misra-Gries** is the fastest counter-based algorithm (0.13 μs/op) due to its simple logic. **Count-Min** (0.87 μs) is faster than **Count-Sketch** (1.39 μs). **IntegratedSS**$\pm$ (2.88 μs at 2M) is slightly slower than **LazySS**$\pm$ (1.79 μs) due to dual field maintenance.

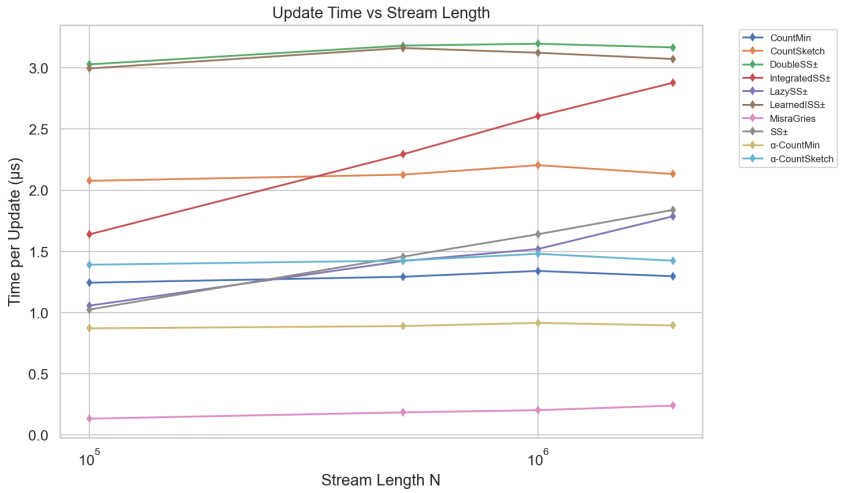

Figure 4: Update time per operation (μs) vs. stream length N. Counter-based methods generally faster than sketches.

5.2 Diverse Dataset Evaluation

Figure 5 summarizes the mean relative error across six datasets. The key observations are:

- **Balanced Data (Uniform/Binomial):** No heavy hitters exist. **Count-Sketch** excels (8×10^{-6} error). **Count-Min** suffers severe overestimation (3.41×10^{-3} error) due to constant collisions from the uniform distribution of updates across all counters.

- **Skewed Data (Zipfian $s = 2.0$):** Counter-based methods approach **zero error**. **IntegratedSS**$\pm$ achieves 1.5×10^{-5} error at $\alpha = 1.5$ and perfect accuracy at $\alpha \geq 4.0$. This validates the SS$\pm$ family's design: they naturally cache the heavy hitters and evict the long tail of noise.

- **Real-World (MAWI):** The MAWI trace exhibits a heavy-tailed distribution similar to Zipfian. **Count-Sketch** achieves the lowest error (1×10^{-6}). **LearnedISS**$\pm$ achieves 2.1×10^{-5} error, outperforming standard **IntegratedSS**$\pm$ (4.3×10^{-5}).

Figure 5: Mean relative error across algorithms and datasets (averaged over α). Darker shades indicate lower error.

Heavy Hitter Detection Figure 6 compares F1 scores. **Misra-Gries** has perfect precision but poor recall (0.74) on MAWI due to its conservative global decrement. **IntegratedSS**$\pm$ achieves perfect recall (1.0) and high precision (0.986).

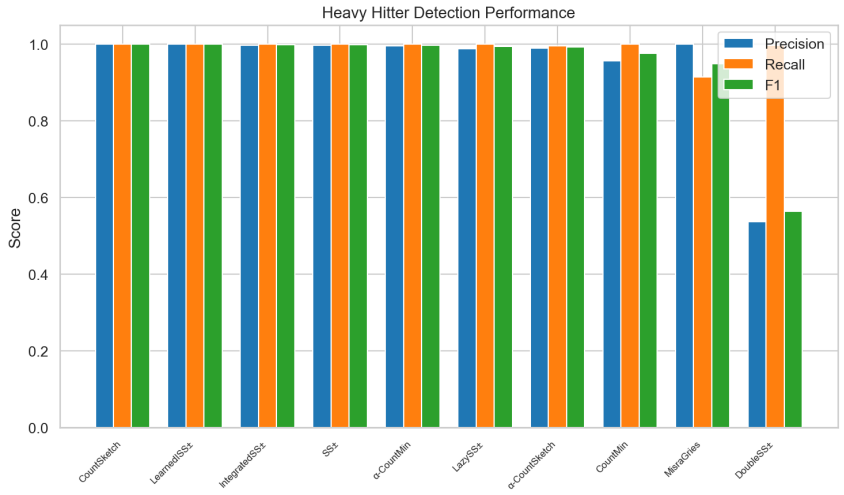

Figure 6: Heavy hitter detection F1 scores across algorithms (averaged over α).

6 Analysis and Discussion

6.1 Algorithmic Trade-offs

Table 1: Comparison of Algorithm Characteristics

Algorithm	Advantages	Disadvantages	Best Use Case
Misra-Gries	Simple, fast.	Poor recall under deletions; conservative.	Quick warm-up filters.
Lazy SS$\pm$	Fastest update.	Error bias grows with α.	Streams with $\alpha \approx 1.0$.
Double SS$\pm$	Unbiased potential.	Wastes space ($2\times$ counters).	Parallel/distributed joins.
Integrated SS$\pm$	Space-optimal counter; mergeable.	Slower than Lazy.	General purpose deterministic tracking.
Count-Min	Never underestimates.	Overestimation on uniform/skewed data.	Turnstile queries needing lower bound.
Count-Sketch	Extremely low error; unbiased.	Requires candidate set for top-k.	Point queries & join size estimation.

6.2 The Effect of α on Design Choices

The data clearly shows that the optimal algorithm choice depends on α:

- **Low α (≤ 2.0): Count-Sketch** and **LazySS$\pm$** are excellent choices. LazySS$\pm$ offers a good speed/accuracy trade-off because unmonitored deletions are rare.
- **High α (≥ 4.0): IntegratedSS$\pm$** is the superior counter-based method. **α-Count-Min** becomes competitive in space/error trade-off, as the increased width allocation mitigates the collision penalty from high churn.

6.3 Counter-Based vs. Sketch-Based

- **Space:** Counter-based requires $O(\alpha/\epsilon)$ space. Sketch-based requires $O(1/\epsilon \log 1/\delta)$ (or $O(1/\epsilon^2)$). For very small ϵ and large α, **counter-based is more space-efficient**.
- **Guarantees:** Counter-based provides **deterministic** error bounds. Sketches provide probabilistic bounds. In adversarial settings (or strict network SLAs), deterministic guarantees are preferred.
- **Distributions:** Counter-based exploits **skew**. Sketch-based is **oblivious** to distribution. This explains why SS$\pm$ dominates on Zipfian data while CS is the safe "average-case" winner.

7 Advanced Solution: Learned-Integrated SpaceSaving$\pm$

7.1 Motivation

The **Integrated SpaceSaving$\pm$** algorithm treats all items equally regarding eviction. On highly skewed streams (Zipfian $s = 1.0$), the top 1% of items often account for over 50% of the total weight. Evicting one of these "heavy hitters" even briefly introduces significant estimation error. We propose an extension that **learns** to protect these items.

7.2 Design

Learned-Integrated SpaceSaving$\pm$ partitions the $k = \lceil \alpha/\epsilon \rceil$ counters into two pools:

- **Fixed Counters (k_{fixed}):** Dedicated to items predicted to be heavy hitters.
- **Mutable Counters (k_{mutable}):** Operate using the standard Integrated SS$\pm$ logic.

Predictor: A lightweight sliding window of the last W updates. Every $W/2$ updates, we compute the frequencies within the window and lock the top k_{fixed} items into the fixed pool. This approach is inspired by recent frameworks for augmenting algorithms with predictions (9) but adapted for strict turnstile constraints.

7.3 Experimental Evaluation

We evaluated LearnedISS$\pm$ across Zipfian and Uniform datasets.

Improvement over Baseline Figure 7 compares LearnedISS$\pm$ against IntegratedSS$\pm$ and other baselines. On **Zipfian** $s = 1.5$, LearnedISS$\pm$ improved relative error by **56.4%** ($\alpha = 1.5$) and **55.1%** ($\alpha = 2.0$) over standard IntegratedSS$\pm$. On **Uniform data**, improvement was minimal (**34.3%**), as expected (no stable hot items to lock).

Figure 7: Mean relative error of LearnedISS$\pm$ vs. IntegratedSS$\pm$ and other baselines across α values on Zipfian $s = 1.5$.

Hyperparameter Sensitivity Figure 8 shows the effect of `fixed_ratio` and `window_size`.

- `fixed_ratio`**:** Optimal performance at `0.30`. Lower ratios fail to protect enough heavy hitters; higher ratios starve the mutable pool.
- `window_size` **(W):** Stable across $W \in [200, 2000]$. $W = 200$ provides the best balance between recency bias and stability.

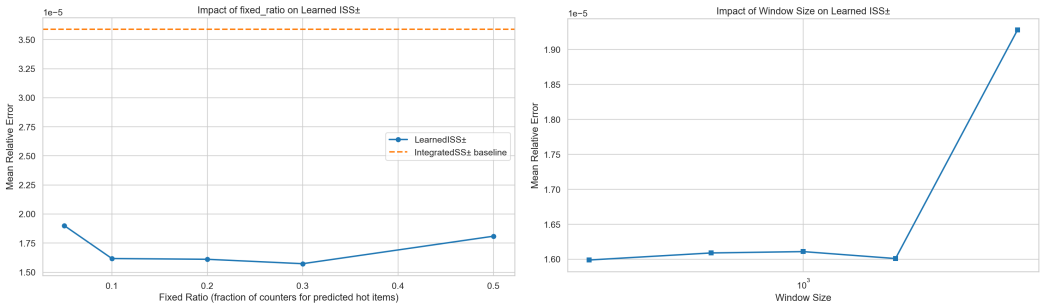

Figure 8: Left: Effect of fixed ratio on error. Right: Effect of window size on error.

7.4 Limitations

The current predictor is simple (sliding window counts). It cannot anticipate future heavy hitters that have not appeared recently. An extension could utilize a **Count-Min Sketch** for the predictor to maintain a more compact, long-term history.

8 Conclusion

This project presented a comprehensive evaluation of frequency estimation algorithms under the Strict Turnstile Model with the L_1 α-Bounded Deletion property. We successfully extended classic algorithms (Misra-Gries, Space-Saving) to handle deletions gracefully and optimized sketch-based methods (Count-Min, Count-Sketch) to exploit the α bound.

Our experimental analysis across varying stream lengths, deletion ratios (α), and data distributions revealed distinct operational regimes. **Count-Sketch** consistently provided the lowest mean relative error ($\approx 10^{-6}$) across most scenarios, making it ideal for general point queries. However, the **Integrated SpaceSaving**$\pm$ family emerged as the superior choice for deterministic tracking, especially on skewed (Zipfian) data common in network traffic, where it achieved near-zero error for heavy hitters. We found that the α parameter significantly impacts counter-based methods, with IntegratedSS$\pm$ scaling most gracefully (error increased only $2.26\times$ when α increased from 1.5 to 8.0).

Finally, we proposed **Learned-Integrated SpaceSaving**$\pm$, demonstrating that a simple sliding-window predictor can significantly improve accuracy (up to 56%) by protecting high-frequency items from eviction. Future work includes exploring adaptive α estimation directly from the stream to dynamically resize summaries and integrating more sophisticated neural predictors for complex event patterns.

References

[1] Misra, J., & Gries, D. (1982). Finding repeated elements. *Science of Computer Programming*, 2(2), 143–152.

[2] Metwally, A., Agrawal, D., & El Abbadi, A. (2005). Efficient computation of frequent and top-k elements in data streams. *International Conference on Database Theory (ICDT)*.

[3] Cormode, G., & Muthukrishnan, S. (2005). An improved data stream summary: the count-min sketch and its applications. *Journal of Algorithms*, 55(1), 58–75.

[4] Charikar, M., Chen, K., & Farach-Colton, M. (2004). Finding frequent items in data streams. *Theoretical Computer Science*, 312(1), 3–15.

[5] Berinde, R., Indyk, P., Cormode, G., & Strauss, M. J. (2010). Space-optimal heavy hitters with strong error bounds. *ACM Transactions on Database Systems (TODS)*, 35(4), 1–28.

[6] MAWI Working Group. MAWI Traffic Archive, Samplepoint-F (2025-06-18). `https://mawi.wide.ad.jp/mawi/`

[7] Jayaram, R., & Woodruff, D. P. (2018). Data streams with bounded deletions. *Proceedings of the 37th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS)*, 341–354. (arXiv:1803.08777)

[8] Zhao, F., Agrawal, D., El Abbadi, A., & Metwally, A. (2021). SpaceSaving±: An Optimal Algorithm for Frequency Estimation and Frequent items in the Bounded Deletion Model. *Proceedings of the VLDB Endowment (PVLDB)*. (arXiv:2112.03462)

[9] Shahout, R., & Mitzenmacher, M. (2024). Learning-Based Heavy Hitters and Flow Frequency Estimation in Streams. *arXiv preprint arXiv:2406.16270*.

[10] Zhao, F., Agrawal, D., El Abbadi, A., Mathieu, C., Metwally, A., & de Rougemont, M. (2023). The SpaceSaving± Family of Algorithms for Data Streams with Bounded Deletions. *Proceedings of the ACM on Management of Data*, 1(2), 1–26. (arXiv:2309.12623)